

OCP etcd Backup/Restore DR Script Suite — Operator Runbook

Companion automation for: [OCP-etcd-backup-restore-testcase-v1_2](#) **Applies to:** OpenShift Container Platform 4.16–4.20, 3-node HA control plane (scale-aware up to ~50 nodes) **Audience:** Cluster administrators executing an etcd disaster-recovery restore, live or as a drill

1. Purpose and scope

This suite automates a single-node-loss etcd disaster recovery on an OpenShift cluster: one control-plane node is designated the **recovery host**, the etcd snapshot is restored there, and the other two masters rejoin the reformed cluster. It wraps the manual steps in the upstream runbook (§1–§8) in five ordered scripts plus a shared library, with logging, a persistent state file, and hard-fail/soft-warn checks at every stage so a run can be audited after the fact.

What this suite does NOT do: take the etcd backup itself (that is a separate, earlier step — see §3 below) and does not touch worker nodes.

2. How the pieces fit together

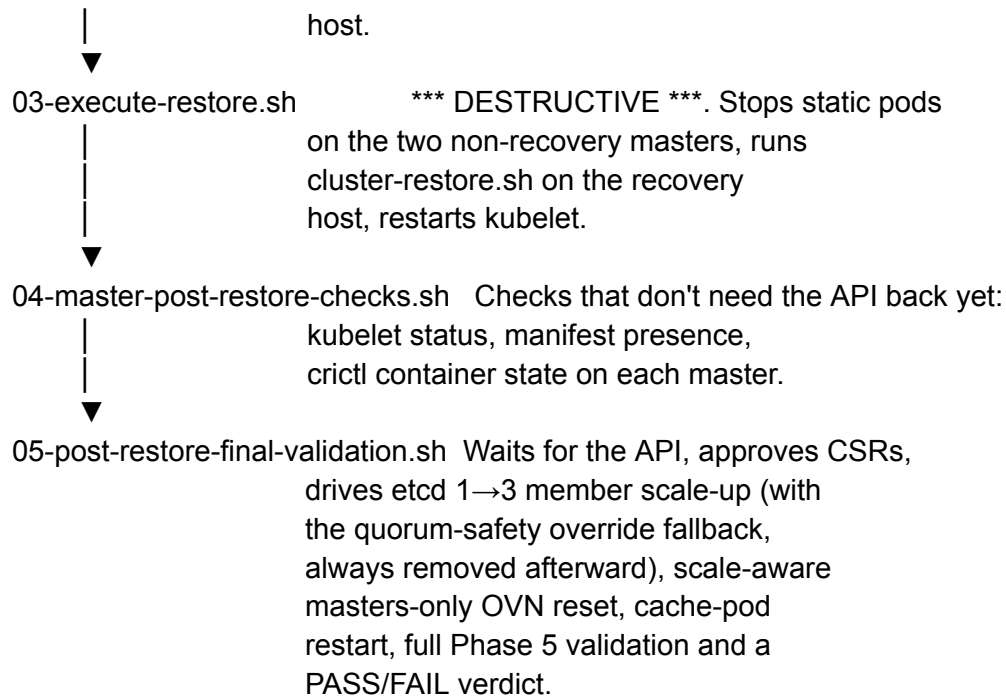
00-common.sh

(sourced by every script — config, logging, SSH helpers, state file)

```
01-preflight-etcd-health.sh      Phase 1: is the cluster healthy enough
    |                            to trust a backup/restore? Picks a
    |                            recovery-host candidate.
```

[take etcd backup — external step, runbook §4]
 [plant Phase-3 DR-drill markers — optional, runbook §5]

02-master-pre-restore-checks.sh	Pre-flight gate: SSH reachable, disk space, cluster-restore.sh present, chrony sync, proxy detection, stages + checksums the backup on the recovery
---------------------------------	---



Everything before script 03 is read-only / staging. Script 03 is the point of no return for API availability — everything from there on is recovery and verification.

3. Before you run anything

3.1 Configure `00-common.sh`

Edit the top of the file for your cluster:

```
SSH_USER="core"
MASTER_NODES=(
  "master1:10.10.0.22"
  "master2:10.10.0.23"
  "master3:10.10.0.24"
)
```

Use **raw IPs**, not FQDNs. This avoids an SSH hostname-doubling failure mode (runbook §B.7) where `$node.cluster.example.com` gets appended to an already fully-qualified name.

Other tunables live here too and rarely need changing:

Variable	Default	Meaning
BACKUP_SRC_DIR	/home/core/assets/backup	Where the backup lives <i>on the node that took it</i>
BACKUP_LOCAL_DIR	/tmp/ocp-backup-<date>	Where the backup lives <i>on the bastion</i>
RESTORE_DEST_DIR	/home/core/backup	Where the backup is staged <i>on the recovery host</i>
OVN_MASTERS_ONLY_NODE_THR ESHOLD	5	Above this node count, OVN reset is masters-only
ETCD_SCALE_TIMEOUT_SEC	900 (15 min)	How long to wait for etcd to reach 3 members before applying the quorum-safety override
OPERATOR_SELFHEAL_TIMEOUT _SEC	900 (15 min)	How long to wait for kube-apiserver/KCM/scheduler to self-heal before force-redeploying them
POLL_INTERVAL_SEC	10	Poll cadence for the above waits

3.2 What you need in hand

- A `cluster-admin` kubeconfig, logged in on the bastion (`oc login`).
- SSH access to all three masters as `core` (or whatever `SSH_USER` is set to), with `sudo`.
- [cluster-restore.sh](#) already present and executable at `/usr/local/bin/cluster-restore.sh` on whichever master becomes the recovery host (script 02 checks this; it does not install it).
- An etcd backup (`snapshot_*.db` + `static_kubernetes_*.tar.gz`) taken **before** the incident and staged in `BACKUP_LOCAL_DIR` on the bastion. Taking the backup itself is outside this script suite — see runbook §4.

3.3 Logging and state

- Every script sources `00-common.sh`, which creates `~/.ocp-etcd-dr/` on the bastion.
- Each run writes a timestamped log to `~/.ocp-etcd-dr/logs/`.

- Shared facts (recovery host, cluster-wide proxy settings, cluster scale) persist across scripts in `~/ .ocp-etcd-dr/dr-state.env`, written with `save_state` and read with `load_state`. This is what lets script 05 know which host was the recovery host without re-deriving it, for example.
 - Every hard failure increments a `FAIL_COUNT` counter (via the `bad` helper); each script's final banner reports that count and the script exits non-zero if it's greater than zero. This makes the whole chain safe to run under `&&` in automation, though step 3 should be run interactively in production given its confirmation prompt.
-

4. `00-common.sh` — shared library

Not run directly; sourced by scripts 01–05. Provides:

- **Config** — `SSH_USER`, `MASTER_NODES`, backup directories, timeouts (see 3.1).
 - **Logging helpers** — `log`, `warn`, `error`, `ok`, `bad`, `die`, `banner`. `bad` is the one that counts toward `FAIL_COUNT`; `die` logs an error and exits immediately.
 - **confirm** — interactive type 'yes' to continue gate, used before destructive actions.
 - **SSH/SCP wrappers** — `ssh_node`, `ssh_node_ok`, `scp_to_node`, `scp_from_node`. All SSH always targets the raw IP, never a derived hostname.
 - **State helpers** — `save_state`, `load_state`, `get_recovery_host`, `non_recovery_nodes`.
 - **get_etcd_pod** — returns the first pod matching `-n openshift-etcd -l app=etcd`, used everywhere an `etcdctl` command needs to run inside a live etcd pod via `oc rsh`.
-

5. Script-by-script reference

5.1 `01-preflight-etcd-health.sh` — Phase 1 pre-flight

Run from: the bastion **Runbook mapping:** §3.1–§3.4 (pre-backup health), §6.1.2 (recovery-host selection) **Destructive:** No

Why it exists: a backup taken against an already-unhealthy cluster is worse than no backup at all — you'd be building your recovery plan on data that doesn't represent a good state. This script is the gate before you even take the backup.

What it checks, in order:

1. **ClusterVersion** is not **Progressing** (an in-flight upgrade should finish before a DR exercise).
2. **All ClusterOperators** report **Available=True**, **Progressing=False**, **Degraded=False**. A benign **node-tuning Progressing=True** is common and called out as a known exception to sanity-check rather than an automatic blocker.
3. **Configured master nodes are Ready**. The first Ready master becomes the **recovery-host candidate** — this is only a candidate; you can override it by hand-editing the state file before running script 03.
4. **etcd member health**, via **etcdctl** run inside a live etcd pod (**oc rsh**): **member list**, **endpoint health --cluster**, **endpoint status --cluster**.
5. **Recent openshift-etcd events** are scanned for quorum-loss, leader-change, or bad-member signals.
6. **Existing backup check** on the recovery-host candidate — warns (does not fail) if **BACKUP_SRC_DIR** already has content, so you don't silently overwrite an older backup without noticing.

Persists to state: **RECOVERY_HOST_NAME**, **RECOVERY_HOST_IP**, **PREFLIGHT_ETCD_POD**, **PREFLIGHT_TIMESTAMP**.

Exit: non-zero if any hard-fail check (**bad**) fired.

After this script: take the actual etcd backup (external to this suite, runbook §4) and, if this is a DR drill, plant the Phase 3 marker resources (runbook §5) that script 05 will later check are gone.

5.2 02-master-pre-restore-checks.sh — pre-restore gate

Run from: the bastion, SSHes into masters **Runbook mapping:** §6.0 (at-scale pre-flight), §6.1.1–§6.1.3 (pre-restore prep), §2.4 (proxy check), §4.6 (file-permission trap) **Destructive:** No — but it does write files (the staged backup) to the recovery host

Prerequisite: script 01 has already run and recorded a recovery host in the state file (or you've exported **RECOVERY_HOST_NAME/IP** yourself).

What it does, against the recovery host unless noted:

1. **SSH reachability to all three masters.** You need all three open *before* the restore starts, because the API becomes unreachable shortly after §6.3 begins and you won't be able to fall back on **oc**.
2. **Disk space** on **/var/lib** and **/home/core**.

3. **cluster-restore.sh** present and executable at `/usr/local/bin/cluster-restore.sh`.
4. **Chrony time sync** — etcd is clock-skew sensitive (runbook §B.4). A non-**Normal** leap status warns but doesn't hard-fail.
5. **Cluster-wide proxy detection** — reads `oc get proxy cluster`. If a proxy is configured, the HTTP/HTTPS/no-proxy values are saved to state so script 03 can export them (with `sudo -E`) when it runs `cluster-restore.sh`, since sudo drops environment variables by default.
6. **Copies and verifies the backup**: copies `snapshot_*.db` and `static_kubernetes_*.tar.gz` from `BACKUP_LOCAL_DIR` on the bastion to `RESTORE_DEST_DIR` on the recovery host, then compares SHA256 sums computed independently on both ends. A mismatch is a hard failure — do not proceed with a restore off an unverified copy.
7. **Cluster scale snapshot** — records total node / master / worker counts. This isn't used here; it's recorded so script 05 can later decide whether the OVN reset should be masters-only.

Persists to state: `CLUSTER_PROXY_HTTP`, `CLUSTER_PROXY_HTTPS`, `CLUSTER_PROXY_NOPROXY` (if applicable), `CLUSTER_TOTAL_NODES`.

Exit: non-zero if any hard-fail check fired — resolve before running script 03.

5.3 03-execute-restore.sh — the destructive step

Run from: the bastion, orchestrates over SSH **Runbook mapping:** §6.2 (stop static pods on the other control-plane nodes), §6.3 (run the restore script on the recovery host), §6.4 (restart kubelet) **Destructive: Yes. This interrupts API availability.**

Requires explicit operator confirmation (`confirm`) before doing anything, and re-verifies SSH to all three masters plus the presence of a staged `snapshot_*.db` on the recovery host before making any change — if either check fails, it aborts without touching anything.

Sequence:

1. **Stop static pods on the two non-recovery masters only** — never on the recovery host itself; `cluster-restore.sh` handles that node. For each of `etcd`, `kube-apiserver`, `kube-controller-manager`, `kube-scheduler`:
 - Moves the static-pod manifest out of `/etc/kubernetes/manifests/` to `/tmp`.
 - Polls `crictl ps` until the corresponding workload container is gone — **guard pods are filtered out** (`grep -v -E 'operator|guard'`) since they're

- expected to keep running per runbook §6.2. Gives up after 120s per component with a warning rather than hanging indefinitely.
- Moves `/var/lib/etcd` aside (timestamped) on that node so it can't rejoin later with stale data — called out in the script as "critical." Progress is saved to state as `RESTORE_STOP_STATIC_PODS_DONE`.
2. **Runs `cluster-restore.sh` on the recovery host.** If a cluster-wide proxy was detected in script 02, `HTTP_PROXY/HTTPS_PROXY/NO_PROXY` are exported before the call and `sudo -E` is used so they survive the privilege escalation. A non-zero exit here is treated as fatal — the script tells you to stop and follow runbook §8 (Rollback/Abort Criteria) and collect a must-gather rather than trying to continue. Progress is saved to state as `RESTORE_SCRIPT_COMPLETED`.
 3. **Restarts `kubelet` on the two non-recovery masters** (`daemon-reload + restart kubelet`). The recovery host's `kubelet` is expected to already have been restarted by `cluster-restore.sh` itself — this script just checks its status for the log rather than restarting it again.

Expected state on exit: only the recovery host has `etcd` / `apiserver` / `controller-manager` / `scheduler` workload pods running; the other two masters are empty of those workloads until the `etcd` operator scales back up to 3 members (that happens in script 05).

5.4 04-master-post-restore-checks.sh — immediate post-restore checks

Run from: the bastion (default, SSH loop over all masters) **or** directly on a master with `--local` **Runbook mapping:** §6.4 (`kubelet` restart verification), §6.5 note, §6.7 (verify single `etcd` member started, at the `crictl` level), §B.1 (Check 1), §B.5 (mirror-pod desync diagnostic)

Destructive: No

Why it exists: the API is typically unreachable for several minutes after `cluster-restore.sh` runs, so this script deliberately avoids `oc` entirely and works off `systemctl` / `crictl` / the filesystem directly on each master over SSH (or locally, in `--local` mode).

Per master, it checks:

1. **`kubelet` is active** (`systemctl is-active kubelet`).
2. **Static pod manifests present in `/etc/kubernetes/manifests/`.** Expectations differ by role:

- On the **recovery host**, all four manifests (`etcd-pod.yaml`, `kube-apiserver-pod.yaml`, `kube-controller-manager-pod.yaml`, `kube-scheduler-pod.yaml`) are expected — their absence is a hard failure.
 - On **non-recovery masters**, they're expected to be *absent* at this point; if any are already present, it's logged as informational (the operator has started scaling that node back up), not a failure.
3. **cricctl workload containers**, per component, with the same `operator|guard` filter used in script 03: expected running on the recovery host, expected absent on the other two (again, logged rather than failed if already present, since that just means the operator is ahead of schedule).
 4. Logs a reminder that the mirror-pod desync condition described in runbook §B.5 (an etcd container running with no Pod IP visible) can only really be confirmed once the API is back, via `oc get pods -o wide` — that check lives in script 05.

Exit: non-zero if any hard-fail check fired. On success, it tells you to wait for the API and proceed to script 05.

5.5 05-post-restore-final-validation.sh — completion + Phase 5 validation

Run from: the bastion **Runbook mapping:** §6.5–§6.10 (drive recovery to completion) and Phase 5 Post-Restore Validation, §7.1–§7.9 **Destructive:** Somewhat — it deletes pods and force-redeploys operators, but only as part of the documented recovery flow, and it's written to be safely re-run if an earlier attempt didn't finish.

Usage:

```
./05-post-restore-final-validation.sh [marker-project-name]
```

If you omit the marker project name, the §7.5 marker-project check is skipped with a warning rather than failing — pass the name of whatever marker project you planted in Phase 3 to get full coverage.

Sequence:

1. **Wait for the API** — polls `oc get nodes` every 10s for up to 10 minutes. If it never comes up, the script tells you to check kubelet on all masters (i.e., re-check script 04) before continuing, and dies.

2. **Approve pending CSRs** — twice, with a short pause between, since a second wave of CSRs commonly appears right after the first is approved.
3. **Force-redeploy etcd** using a uniquely-suffixed `forceRedeploymentReason` (`date + uuidgen/$RANDOM`). The unique suffix matters — reusing the same reason string is a documented silent no-op trap (runbook §B.6).
4. **Poll for etcd to scale from 1 → 3 members**, checking both that 3 etcd pods report `5/5 Running` and that `etcdctl member list` shows 3 members in `started` state. Waits up to `ETCD_SCALE_TIMEOUT_SEC` (default 15 min).
 - **If it doesn't reach 3 members in time**, the script applies the **quorum-safety override** (`unsupportedConfigOverrides.useUnsupportedUnsafeNonHANonProductionUnstableEtcd: true`) — noted as having been needed on 2 of 3 documented production runs — force-redeploys etcd again, and waits once more.
 - **If it still isn't at 3 members after the override**, the script stops (does not keep retrying) and tells you to escalate per runbook §8 and collect a must-gather (`oc adm must-gather`). This is a hard abort, not a soft warning.
 - **Regardless of which path was taken**, once etcd is healthy the script **always** removes the override if it applied one, and then independently re-reads `unsupportedConfigOverrides` to confirm it's actually empty — flagging it explicitly if it isn't, since this setting must never be left on permanently.
5. **Wait for kube-apiserver / kube-controller-manager / kube-scheduler to self-heal**, up to `OPERATOR_SELFHEAL_TIMEOUT_SEC` (default 15 min). Any still degraded after that window get individually force-redeployed (same unique-reason pattern) rather than left alone.
6. **Scale-aware OVN-Kubernetes reset** (v1.2 revision of the runbook). Uses the node count recorded by script 02 (`CLUSTER_TOTAL_NODES`) against `OVN_MASTERS_ONLY_NODE_THRESHOLD`. In the current implementation this script **always restarts OVN only on masters**:
 - Removes the OVN central DB files on each master over SSH (tries the `ovn-ic` path first, falls back to the legacy `ovn` path).
 - Deletes `ovnkube-control-plane` pods and waits 15s.
 - Deletes `ovnkube-node` pods, but only those scheduled on master nodes, pausing 15s between each.
 - Worker `ovnkube-node` pods are deliberately left alone — they don't hold central DB state — and are only restarted reactively, by hand, if they show failures (§6.9.4).

7. **Restart pods that cache API state:** ingress, image-registry (only if `managementState=Managed` — a no-op on bare-metal by design, §B.3), DNS, Prometheus, Alertmanager, the authentication operator, and the OAuth API server.
8. **Phase 5 — Post-Restore Validation (§7.1–§7.9):**

§

Check

- 7.1 `oc get clusterversion` reconciled cleanly (no reconcile errors)
 - 7.2 All ClusterOperators `Available=True / Progressing=False / Degraded=False`
 - 7.3 All nodes `Ready`
 - 7.4 `etcd member list + endpoint health --cluster` clean
 - 7.5 Marker project (if supplied) **and** marker ConfigMap `post-backup-marker` in `default` are both gone — proof the restore actually reverted state, not just that it ran
 - 7.7 No lingering pending CSRs (approves any found)
 - 7.8 `unsupportedConfigOverrides` confirmed empty (final re-check)
 - 7.9 Firing Prometheus alerts, reported informationally — not a pass/fail gate, since some settle within ~15 minutes
9. **Verdict:** prints `TEST RESULT - <N> hard failure(s). FAIL_COUNT == 0 → PHASE 5 VALIDATION: PASS`, exit 0. Otherwise `FAIL`, exit 1, pointing you at the log and runbook Appendix B.

6. Order of operations (quick reference)

```
oc login <cluster>                # cluster-admin kubeconfig

./01-preflight-etcd-health.sh      # health check, picks recovery host
# --- take the actual etcd backup here (runbook §4), land the two files
#   in $BACKUP_LOCAL_DIR on the bastion ---
# --- if this is a DR drill, plant Phase 3 markers here (runbook §5) ---
```

```

./02-master-pre-restore-checks.sh    # pre-restore gate + stages backup

./03-execute-restore.sh             # DESTRUCTIVE — confirms before running

./04-master-post-restore-checks.sh   # immediate crictl/kubelet-level checks
# or, directly on a master:
#  scp 00-common.sh 04-master-post-restore-checks.sh core@master1:/tmp/
#  ssh core@master1 '/tmp/04-master-post-restore-checks.sh --local'

./05-post-restore-final-validation.sh <marker-project-name>

```

Scripts 01, 02, and 04 exit non-zero on hard failure, so the non-destructive portion is safe to chain with `&&`. Because script 03 is destructive and interactive by design (it prompts for `yes`), running the suite fully unattended is not recommended for production restores.

7. Design notes and gotchas carried over from the runbook

- **Quorum-safety override**
(`useUnsupportedUnsafeNonHAProductionUnstableEtcd`) is applied only as a fallback, only if etcd doesn't reach 3 members within `ETCD_SCALE_TIMEOUT_SEC` (default 15 min) — and script 05 unconditionally verifies afterward that it's been removed, independent of which code path set it.
- **OVN reset is masters-only by design** (v1.2 revision). An earlier restart-every-node approach caused a stuck `Terminating` pod on a 35-node production run; workers are handled reactively instead.
- **`forceRedeploymentReason` patches always use a unique suffix** (`date + uuidgen`) to avoid a silent no-op if the same reason string is reused (§B.6).
- **Static-pod "stopped" checks filter `operator|guard` pods** — guard pods are expected to keep running throughout (§6.2), so their presence doesn't mean the stop failed.
- **Raw IPs everywhere for SSH**, never derived hostnames — avoids an FQDN-doubling failure (§B.7).
- **Recovery-host selection is a candidate, not final** — script 01 picks the first Ready master; you can override it by editing the state file (`~/ .ocp-etcd-dr/dr-state.env`) before running script 03.
- **§7.5 (marker resources) is the closest thing to proof-of-restore** — a cluster can look entirely healthy (§7.1–§7.4, §7.8) while the restore itself silently no-op'd; the marker

check is what actually confirms state was reverted, which is why it's worth planting a marker even outside of formal DR drills if you want that assurance.

8. Where to look when something fails

Symptom	Where to look
API never comes back after script 03	Runbook §6.4/§6.5; re-run script 04 to check kubelet on all masters
Static pod won't stop within 120s (script 03)	Investigate manually on that node before continuing — the script only warns and moves on
<code>cluster-restore.sh</code> exits non-zero	Runbook §8 Rollback/Abort Criteria; collect <code>oc adm must-gather</code>
etcd stuck at 1 member even after the override (script 05)	Runbook §8; script 05 aborts automatically and tells you to collect must-gather
<code>unsupportedConfigOverrides</code> still set after script 05	Remove manually — do not call the run a PASS until confirmed empty
Pod present but no Pod IP / mirror-pod desync	Runbook §B.5
Missing static-pod manifest / "Missing operand"	Runbook §B.1
Chrony leap status not <code>Normal</code>	Runbook §B.4 — etcd is clock-skew sensitive
Everything green except the restore didn't seem to actually happen	Check §7.5 marker output specifically — this is the one check that proves state reversion rather than just cluster health